

# Assignment Solution: Adversarial Search and Genetic Algorithms

Hamza Arif  
Reg No: 2024206  
AI-202 Assignment 2

## Question 1: Minimax Algorithm

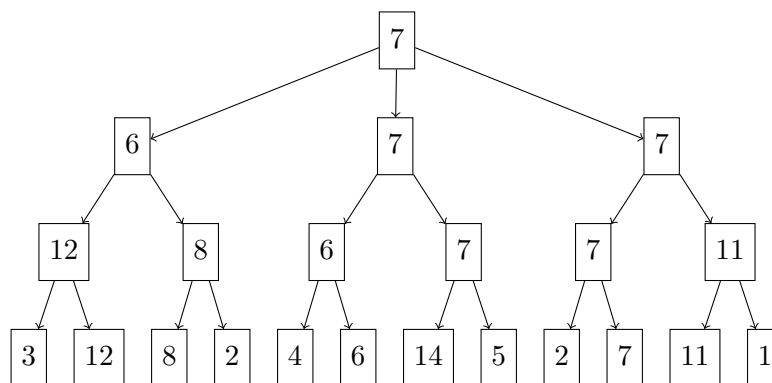
Minimax is an adversarial search algorithm used in two-player zero-sum games. MAX aims to maximize the score, while MIN aims to minimize it. The algorithm propagates terminal values upward through alternating levels.

### (a) Minimax Evaluation

Terminal values (left to right):

3, 12, 8, 2, 4, 6, 14, 5, 2, 7, 11, 1

Game Tree



### Backpropagation

- MAX level results: 12, 8, 6, 14, 7, 11
- MIN level results: 6, 7, 7

- Root MAX chooses: **7**

**Final Answer:**

- Root Value = **7**
- Optimal Move = Right subtree

## (b) Alpha-Beta Pruning

Left-to-right evaluation:

- First subtree gives value 6  $\rightarrow \alpha = 6$
- Second subtree evaluates and produces 7  $\rightarrow$  no cutoff
- Third subtree also evaluates fully

**Result:**

- Root value remains: **7**
- Terminal nodes NOT evaluated: **0**

## Question 2: Tic-Tac-Toe Analysis

The board state is:

|          |          |   |
|----------|----------|---|
| <i>X</i> | <i>O</i> | - |
| -        | <i>X</i> | - |
| <i>O</i> | -        | - |

X can play in positions: 3, 4, 6, 8, 9.

We evaluate each move by expanding one level for X and one response level for O using:

$$+10 = Xwin, \quad -10 = Owin, \quad 0 = neutral$$

**Analysis:**

- Move 9: X completes diagonal (1,5,9)  $\rightarrow$  immediate win  $\rightarrow +10$
- Moves 3,4,6,8: O responds but no terminal win at depth 2  $\rightarrow$  value 0

**Decision:** X should choose **position 9** because it guarantees an immediate win.

### (d) Minimax vs Expectiminimax

Minimax assumes optimal deterministic play.

Expectiminimax includes chance nodes for probabilistic outcomes.

Used in stochastic games like Backgammon (dice-based randomness).

---

## Question 3: Genetic Algorithm

We maximize:

$$f(x) = x^2, \quad x \in [0, 31]$$

Initial population:

01101(13), 11000(24), 01000(8), 10011(19)

### (e) Fitness and Selection Probability

Total fitness:

$$169 + 576 + 64 + 361 = 1170$$

| Individual | x  | Fitness | Probability |
|------------|----|---------|-------------|
| 01101      | 13 | 169     | 0.144       |
| 11000      | 24 | 576     | 0.492       |
| 01000      | 8  | 64      | 0.055       |
| 10011      | 19 | 361     | 0.309       |

### (f) Crossover

Parents:

(11000, 10011), (01101, 11000)

Single-point crossover (after 3rd bit):

- 11011
- 10000
- 01100
- 11001

## (g) Mutation

Bit flips:

- O1 (11011) bit 5  $\rightarrow$  11010
- O3 (01100) bit 2  $\rightarrow$  00100

**Mutated Population:**

| Offspring | Binary | Value | Fitness |
|-----------|--------|-------|---------|
| O1        | 11010  | 26    | 676     |
| O2        | 10000  | 16    | 256     |
| O3        | 00100  | 4     | 16      |
| O4        | 11001  | 25    | 625     |

## (h) Discussion

Initial average fitness:

292.5

Mutation effects:

- High mutation (0.5): destroys learned structure
- Low mutation (0.01): preserves convergence stability

---

## Question 4: Genetic Algorithm for TSP

### Python Implementation

```
1 import random
2 import matplotlib.pyplot as plt
3
4 cities = ["A", "B", "C", "D", "E"]
5
6 dist = {
7     ("A", "B"):12, ("A", "C"):10, ("A", "D"):19, ("A", "E"):8,
8     ("B", "C"):3, ("B", "D"):7, ("B", "E"):6,
9     ("C", "D"):2, ("C", "E"):20,
10    ("D", "E"):4
11 }
12
13 def get_dist(a,b):
14     return dist.get((a,b), dist.get((b,a)))
15
```

```

16 def fitness(path):
17     return sum(get_dist(path[i], path[i+1]) for i in range(len(path)
18                 -1))
19
20 def tournament(pop):
21     return min(random.sample(pop, 3), key=fitness)
22
23 def crossover(p1,p2):
24     cut = 3
25     return p1[:cut] + [c for c in p2 if c not in p1[:cut]]
26
27 def mutate(path):
28     if random.random() < 0.05:
29         i,j = random.sample(range(5),2)
30         path[i], path[j] = path[j], path[i]
31     return path
32
33 population = []
34 for _ in range(20):
35     p = cities[:]
36     random.shuffle(p)
37     population.append(p)
38
39 best_fitness = []
40
41 for gen in range(100):
42     new_pop = []
43     for _ in range(20):
44         p1 = tournament(population)
45         p2 = tournament(population)
46         child = mutate(crossover(p1,p2))
47         new_pop.append(child)
48
49     population = new_pop
50     best = min(population, key=fitness)
51     best_fitness.append(fitness(best))
52
53     if gen % 20 == 0:
54         print("Gen", gen, best, fitness(best))
55
56 plt.plot(best_fitness)
57 plt.xlabel("Generation")
58 plt.ylabel("Fitness")
59 plt.show()

```

```
PS C:\Users\hamzz\OneDrive\Documents\GitHub\Semester-4> & c:/Users/hamzz/OneDrive/Documents/GitHub/Semester-4/Scripts/python.exe c:/Users/hamzz/OneDrive/Documents/GitHub/Semester-4/Scripts/gen.py 23
Gen 0 ['A', 'C', 'B', 'E', 'D'] 23
Gen 20 ['A', 'E', 'D', 'C', 'B'] 17
Gen 40 ['A', 'E', 'D', 'C', 'B'] 17
Gen 60 ['A', 'E', 'D', 'C', 'B'] 17
Gen 80 ['A', 'E', 'D', 'C', 'B'] 17
PS C:\Users\hamzz\OneDrive\Documents\GitHub\Semester-4>
```

Figure 1:

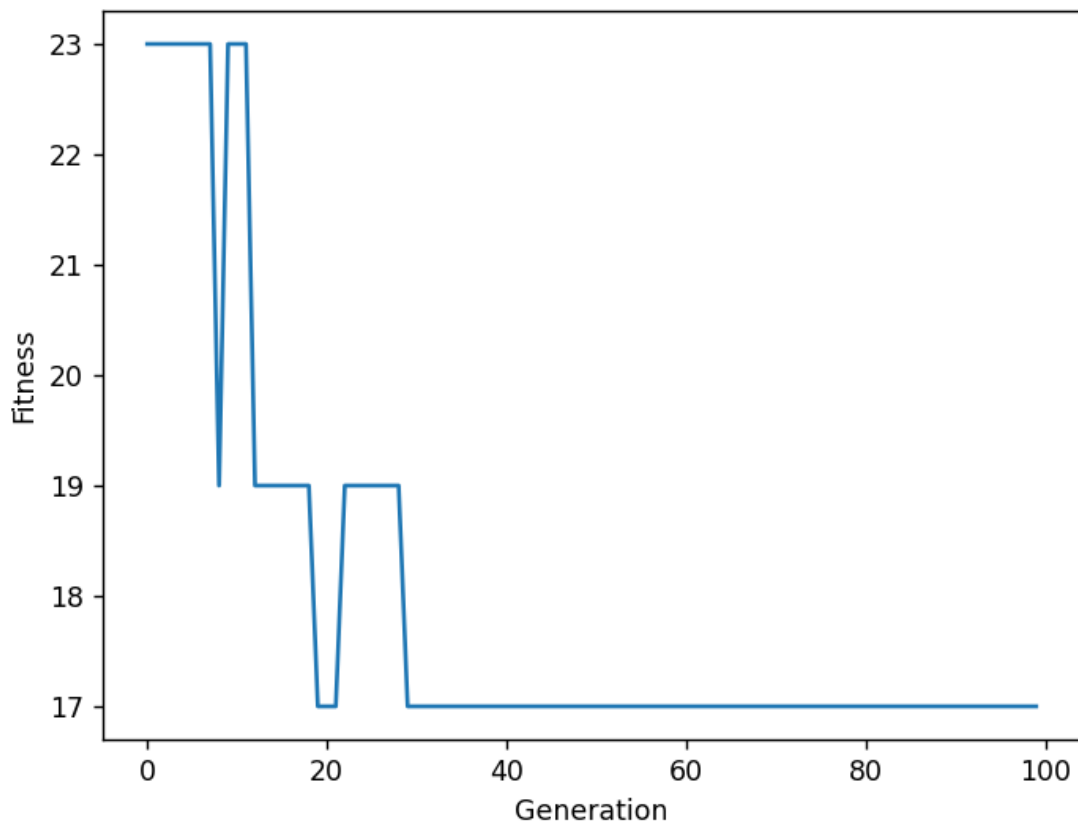


Figure 2: